A setup using Blender Geometry Nodes, to allow a base 3D map based on a flat plane (equirectangular projection, not web mercator), such as one depicting a 3D heightmap of the earth or a fantasy world, to be projected onto a spherical representation. For editing in on a planar, more intuitive map, while allowing for easy representation of the imagined reality of a spherical world.

Also including measuring distance and drawing routes between points, both loxodromic (straight on the flat mercator representation) and orthodromic (shortest actual route between two points on the sphere, but curved on the flat representation).

Optional additional features may include:

- Projection from mercator to other projection schemes
- Projection from globe to plane

No Generative AI used in the project

Spherical Projection, Equirectangular

- Sources and inspiration
  - https://en.wikipedia.org/wiki/Equirectangular_projection
  - https://en.wikipedia.org/wiki/Spherical_coordinate_system
  - https://www.reddit.com/r/blender/comments/1e5q292/geometry_nodes_if_statements_help/

Not very complex once you know what you're doing.

Take the position of every individual vertex of the geometry, and first convert it into latitude and longitude (but in radians, rather than degrees), with negative latitude being points south of the equator, and negative longitudes being west of the Greenwich meridian.

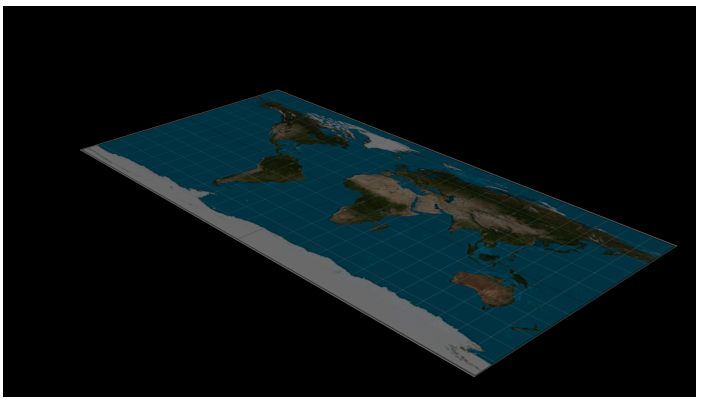
Then, we use this formula, taken from wikipedia:

> Conversely, the Cartesian coordinates may be retrieved from the spherical coordinates (*radius* r, *inclination* θ, *azimuth* φ), where $r \in [0, \infty)$, $\theta \in [0, \pi]$, $\varphi \in [0, 2\pi)$, by
>
> $$x = r \sin\theta \cos\varphi,$$
> $$y = r \sin\theta \sin\varphi,$$
> $$z = r \cos\theta.$$

To put every point of the planar representation where it goes on the spherical representation. BUT while azimuth and longitude are essentially the same, inclination and latitude are very different. So I solve that using the sign maths operator and a bit of a workaround so that the specific case where the latitude is zero works as intended.

Working example

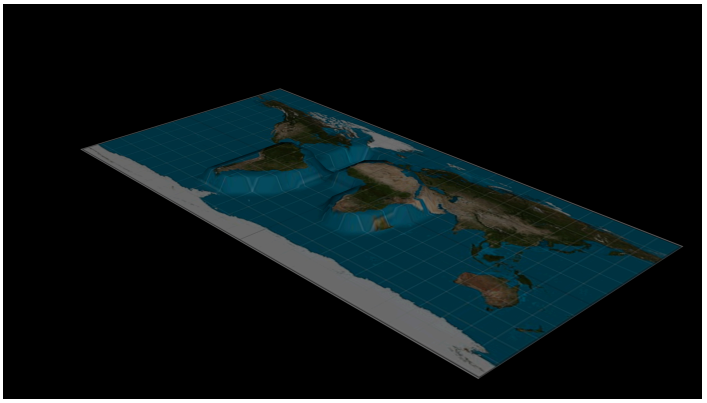
Without modifier



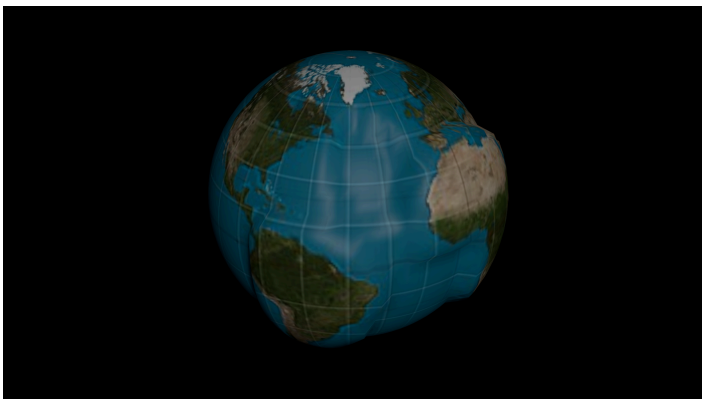
With modifier

Example with 3D features on plane

Without modifier



With modifier



Mercator Projection

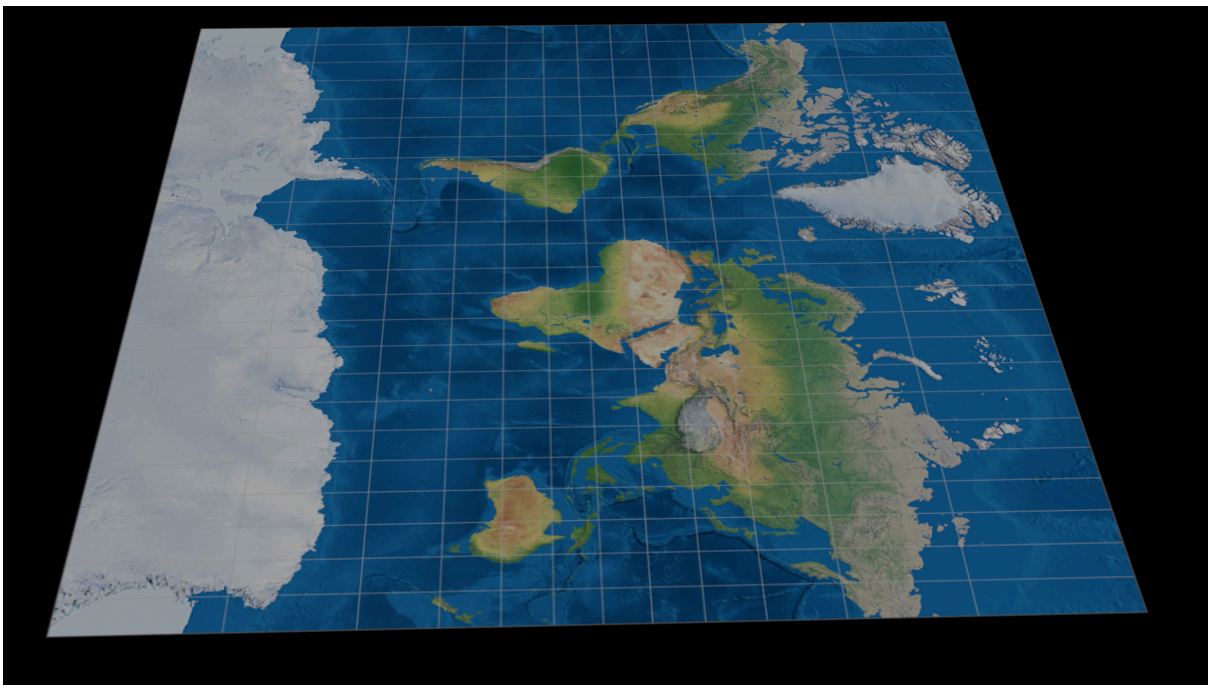
- Sources and inspiration
    - https://en.wikipedia.org/wiki/Mercator_projection

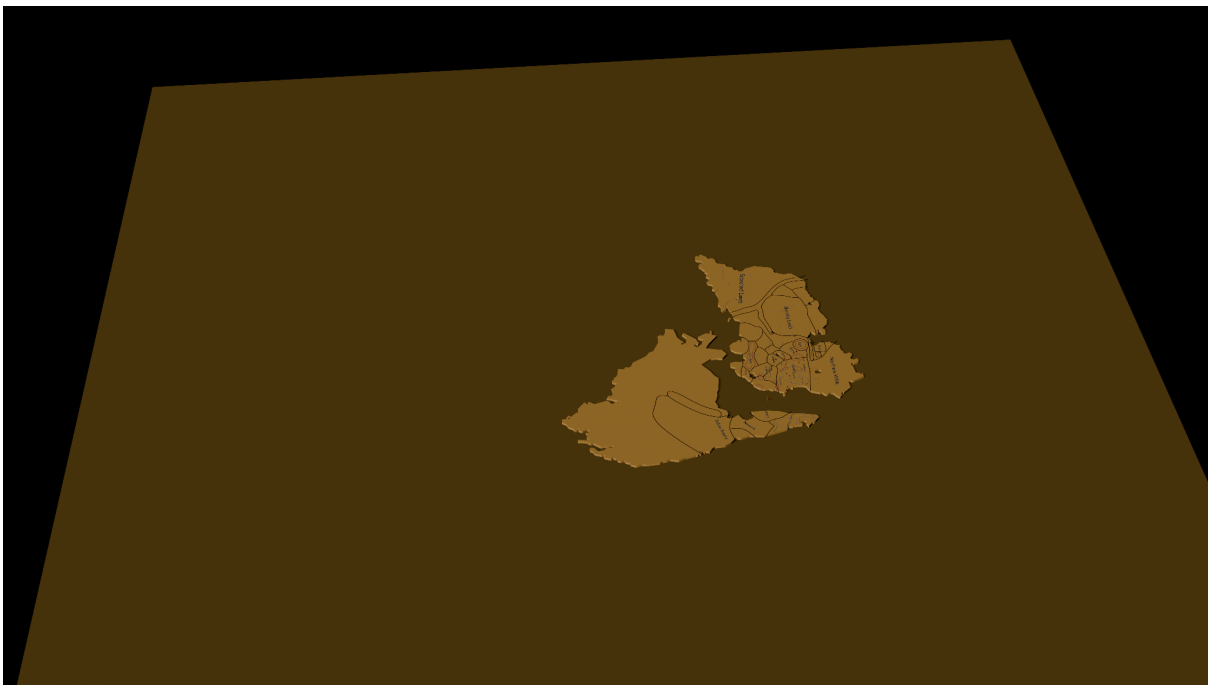
Based on tools developed and what I learned from the equirectangular projection method, I fairly easily developed a way to do the same thing for a mercator projection, preferable because the mercator projection actually represents straight lines on the map as rhumb lines with constant heading on the sphere.

Still something of a work in progress because the coordinates aren't exactly accurate (see the next section about rhumb line distance measurement).

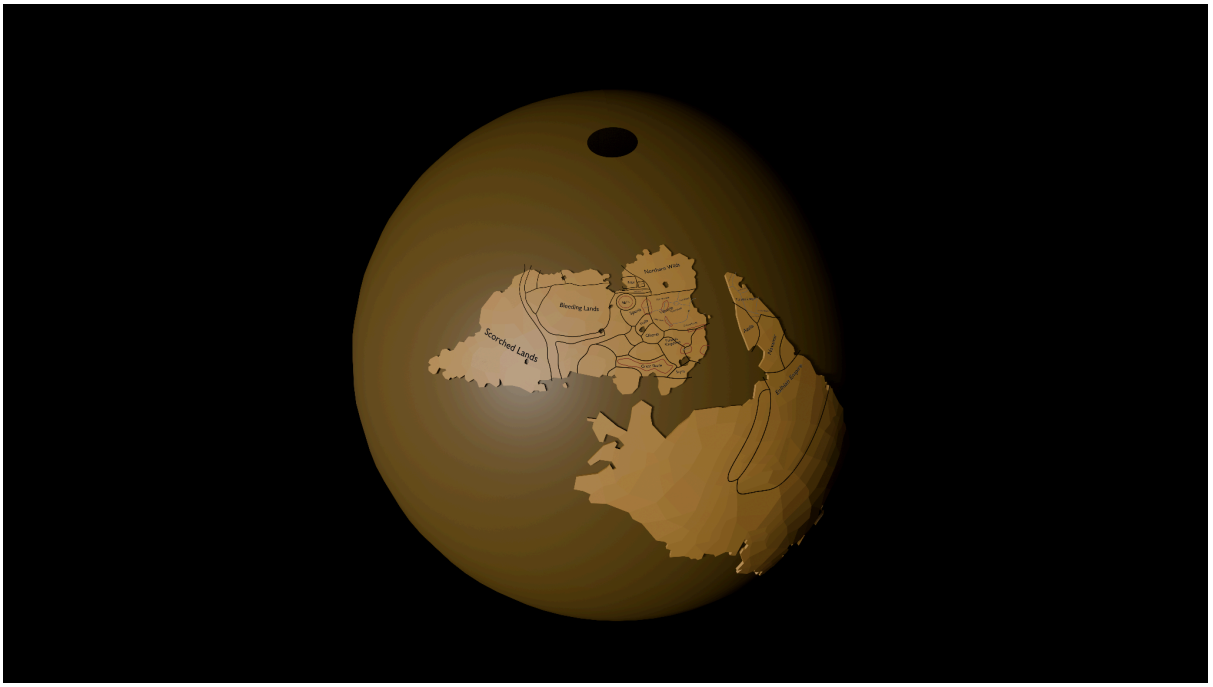
Working example

Without Modifier





With Modifier

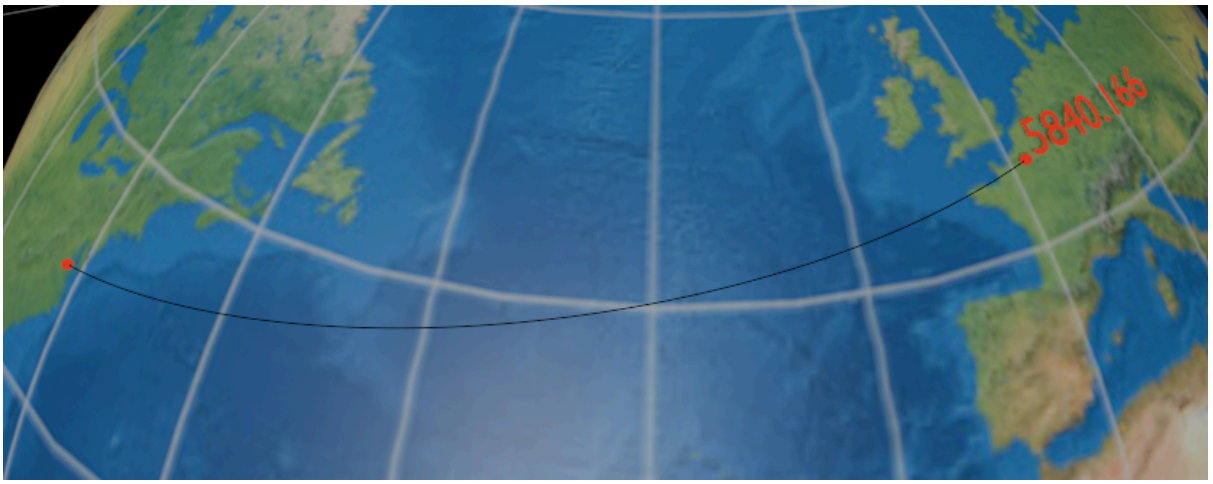


Mercator Rhumb Line Measurement and Ruler

- Sources and inspiration
    - https://www.vcalc.com/wiki/rhumb-line-distance

Rhumb lines are the distance between points measured along a line of constant bearing. This tool measures such a distance between any two points, given by coordinate, on the globular map.

Line between the approximate location of New York and London, with the expected rhumb line distance.



Same line on the flat representation.

—Rhumb Line Distance

(LT1)Latitude 1

51.51 (°) degree angle

(LN1)Longitude 1

0.13 (°) degree angle

(LT2)Latitude 2

40.71 (°) degree angle

(LN2)Longitude 2

-74.01 (°) degree angle

Share Result

5840.5233156 (km) kilometer

Still a work in progress on the matter of the mercator projection of the earth, as stated, because (as you can see) the points with the coordinates of London and New York are slightly off from where those cities should be. But the distance between them is accurate to what can be expected.

The less than 0.5 km difference between the calculator and my system is likely because it uses a slightly different, perhaps not rounded, version of the mean Earth radius (assumed to be 6371 by my system, which is the number most sources including wikipedia gives).

Mercator Orthodromic Measurement and Ruler

- Sources and inspiration
    - https://en.wikipedia.org/wiki/Great-circle_distance

- https://www.charlespetzold.com/blog/2010/06/Great-Circle-Arcs-on-the-Sphere.html
- https://www.cuemath.com/geometry/angle-between-vectors/
- https://en.wikipedia.org/wiki/Mercator_projection

Finding the distance between two points on a globe by the actual shortest distance, the great circle arc connecting them, is not something I found particularly challenging, because there are easily found formulas for that, such as the one on the wikipedia pages in the sources and inspiration for this section.

The surprisingly troublesome part is drawing the line connecting them on a Mercator projection of the globe. For this, I started out by defining two potential approaches. One being to draw the line between the two directly on the Mercator projection, perhaps by finding a method which might be used to navigate along the great circle in real life, since that should fundamentally be solving the same problem: Decide the line to follow to get where you are going. The other being to find a way to define the path in spherical coordinates, i.e. technically defining it on the great circle, and then projecting it down onto the mercator.

I decided to start by exploring the second option. With the specific reasoning that I likely need to know the length of the line to draw it, and while the length on the mercator projection is of little consequence and thus might be hard to find an existing definition for, the great circle definition for the distance is (as mentioned) very easy to find.

Curve definition of the great circle distance

After a significant amount of looking around, I found the idea which I thought might be the key to cracking this, which I had missed since I'd been very focused on looking at the problem in spherical terms, without much personal knowledge of advanced spherical geometry.

While I found defining the great circle in spherical coordinates quite hard, doing so in cartesian coordinates is easier, because we can very easily find the plane containing the great circle arc, since it is also the plane containing the connected points as well as the center of the sphere. So we define this plane, which is trivial in cartesian coordinates, and then constrain our space to only the points within a set distance of the center, giving us the circle. We can potentially solve this by finding a parametrization of the circle using this constraint, for which I would need to repeat some math I've learned previously. But then I had another idea on how to find this circle.

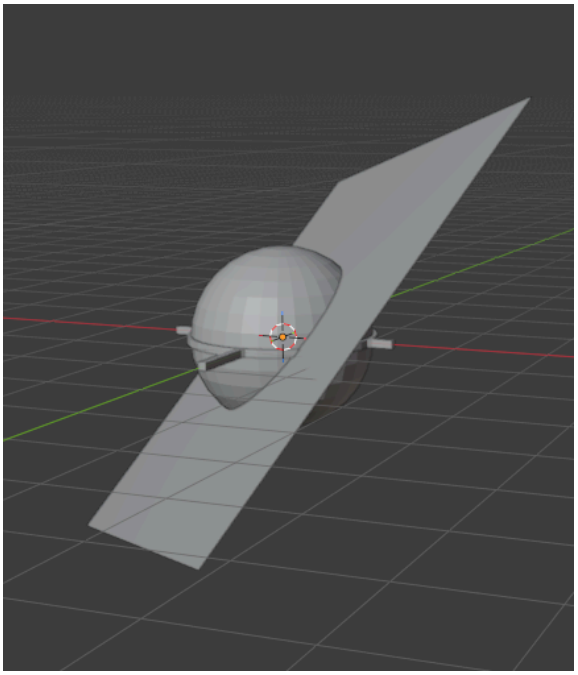
Transforming the Circle

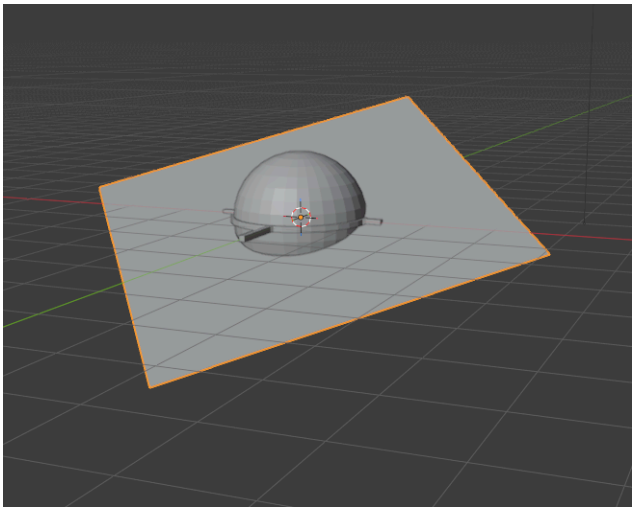
Rather than using the plane and constraint, what if we keep track of our plane, and define a circle around the equator (which is a great circle)? Then, we choose two points on opposite sides of this circle, which are both located on the X axis, and keep track of them. We then rotate the circle about the Y axis, until one of the points is on the plane. Since the two of them and the center of the circle, which is the center of the sphere, form a line, and two of

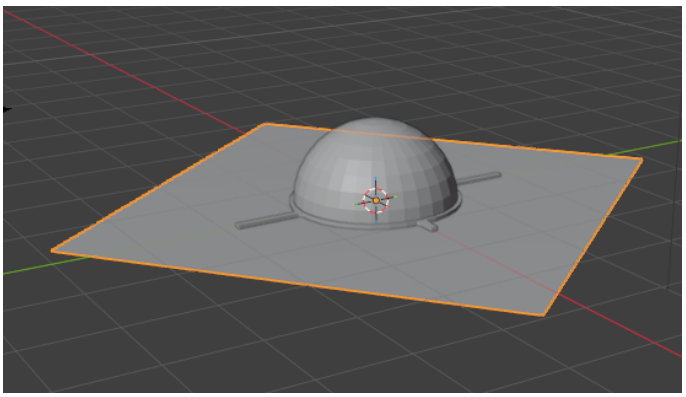
the points on that line fall on the plane, the third must also, meaning all three points fall on the plane.

Now, we can rotate the great circle about the line formed by our two chosen points. Since we are rotating about this line, the points ON that line will stay put, still located on the plane. We now define two points on opposing sides which were originally located on the Y axis, i.e. on our original axis of rotation. Yet again we rotate until one of them falls on the plane, and yet again both must if one does. With this done, we have transformed a circle such that it lies on the plane, meaning it must be the great circle along which we want to traverse.

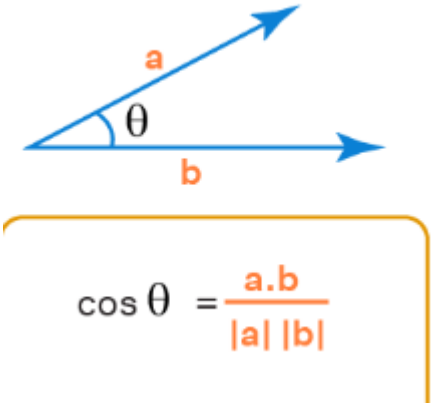
Now we just want a mathematical framework to do this. Although, to make things much easier, we calculate the rotations we need to perform, but do them in reverse, i.e. we rotate the plane about the Y axis so that the X axis points fall on it, and then the same for the Y axis points.







So, first, we calculate the rotation about the Y axis. Since our two points lie on the X axis, we can constrain away the whole Y dimension of this problem, giving us a 2D XZ plane, where we must find the angle we need to rotate a single line (the plane constrained to just the XZ plane) to lie on the X axis. This is also where I need to acknowledge that for any of these rotations there will be two different ways of accomplishing the task, one longer and one shorter rotation (usually), since there are two angles between each of the lines, but I'll figure out the basic procedure and then adjust for this fact.



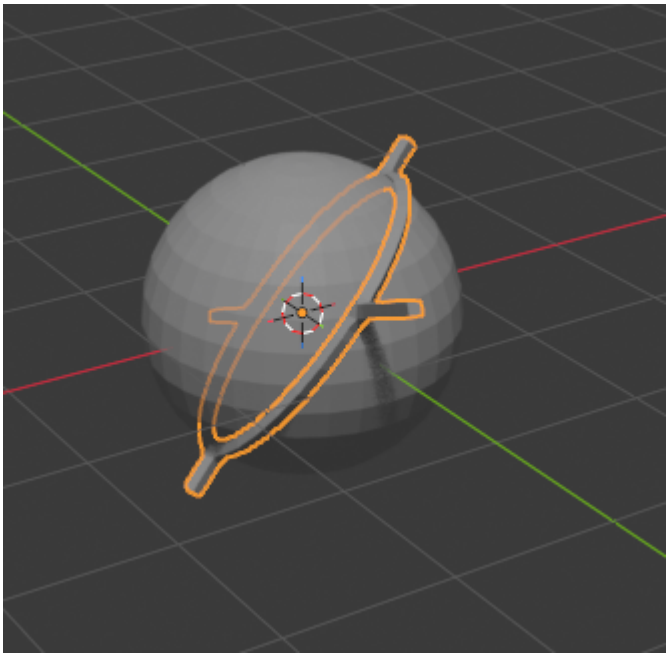
To get the angle by which we must rotate the plane, we simply figure out the angle between the X axis and the line (the line being the restricted version of the plane), which is as easy as picking any point on the X axis, and any point on the line, and using this formula.

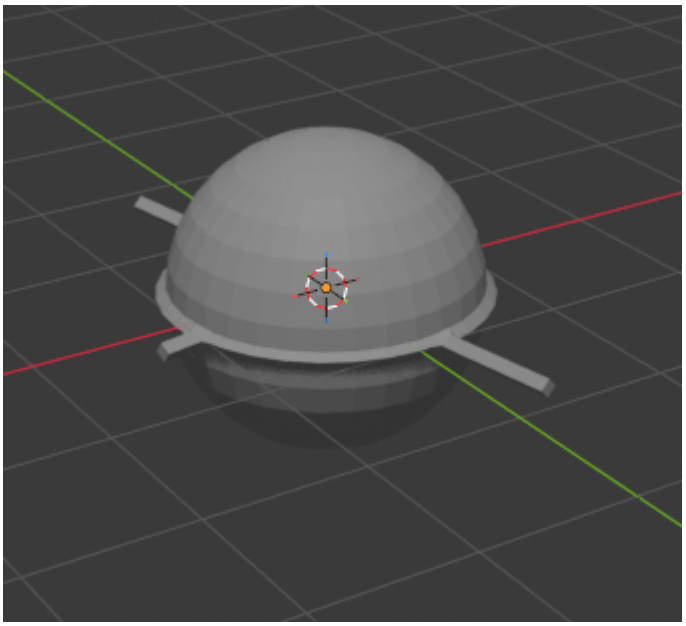
So, since we are defining our plane by 3 points, more specifically by the formula **a** + u(**b** - **a**) + v(**c** - **a**), where u and v are any scalar values, **a** is the center of the sphere (which is also at the origin, for simplicity), and **b** and **c** are our two points on the surface of the globe, we can find the normal to the plane by the cross product between **b** and **c**. This gives us normal **n**, for which we need to find a vector **v** which is orthogonal to **n** and has **v**y = 0. We can just set **v**x = 1 and solve for this quite easily (there may be some cases where this breaks, namely when **nz** is 0, but we can handle those later on).

So we do this, having found **v**, and then find the angle between it and any point on the x axis, so we'll pick this as **x** = (1,0,0) for simplicity.

We find the angle, and then just do a very similar thing to find the angle by which we must rotate about the y axis.

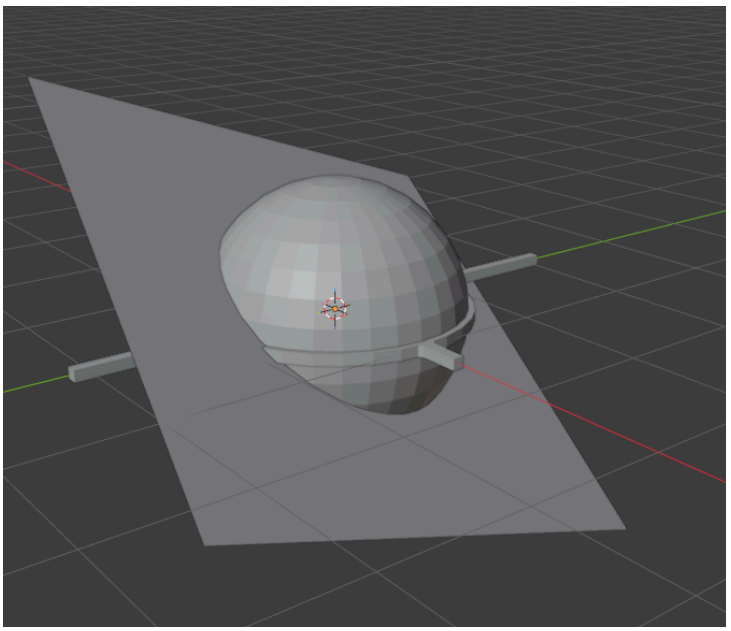
Now, we have a concrete way to rotate the plane onto the XY plane, or rotate the circle onto the plane. I'll implement this to rotate our circle, using geometry nodes, just to make sure it works.

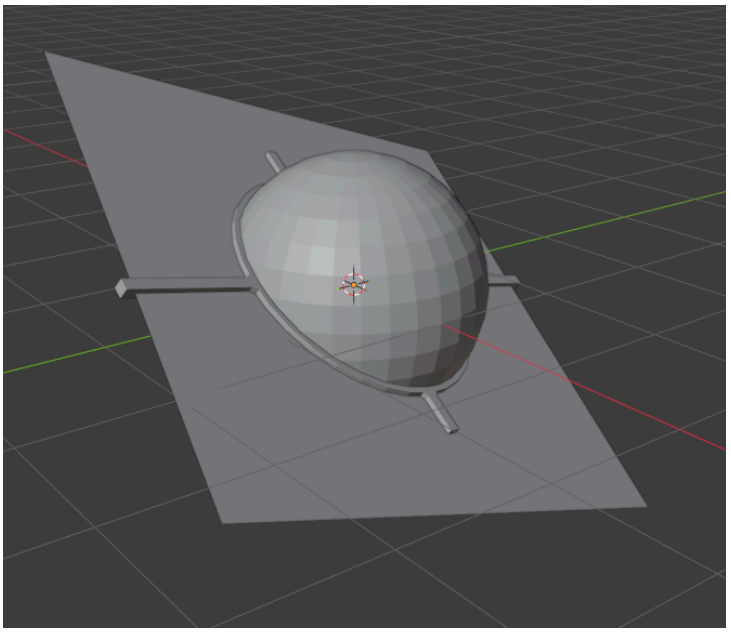




It took a while, but I managed to get it rotating from any plane to the XY plane, using two points on the original plane as input. Now, to invert it, I just need to use the same points as input and still rotate, only doing the opposite, i.e. opposite order, opposite sign.

It took a little bit of trying, mostly just dividing the existing nodes into groups with clear names to sort out the mess and make it clear what needed to happen. But I got it working!



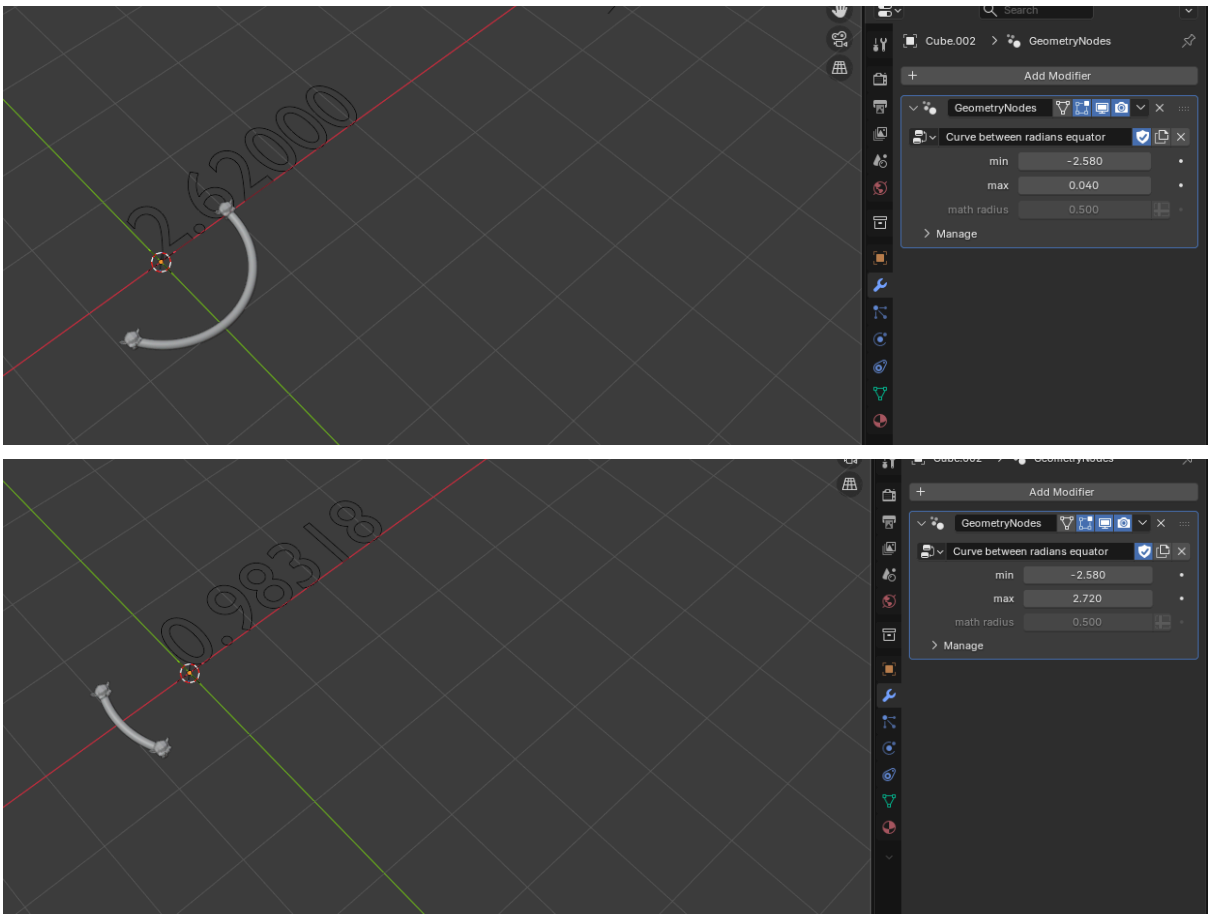


So the rotation works in both directions, which means I have simplified the problem of drawing a curve between two points along any great circle, to drawing one between any two points on the equator, which is a vast improvement.

Drawing the curve on the equator

Alright, with the problem heavily simplified, I can even implement my own way of calculating things because it is just so simple. Because both points in between which I want to draw my arc are now on the plane, they both have the same incline, in terms of spherical coordinates. The distance is then literally just the angle between them! Of course there are two possible, but I can just take the shortest!

So with some switches and such I got the following working, with 2 monkeys to mark the points, their position described in radians, with a line and measurement of the shortest distance between them.

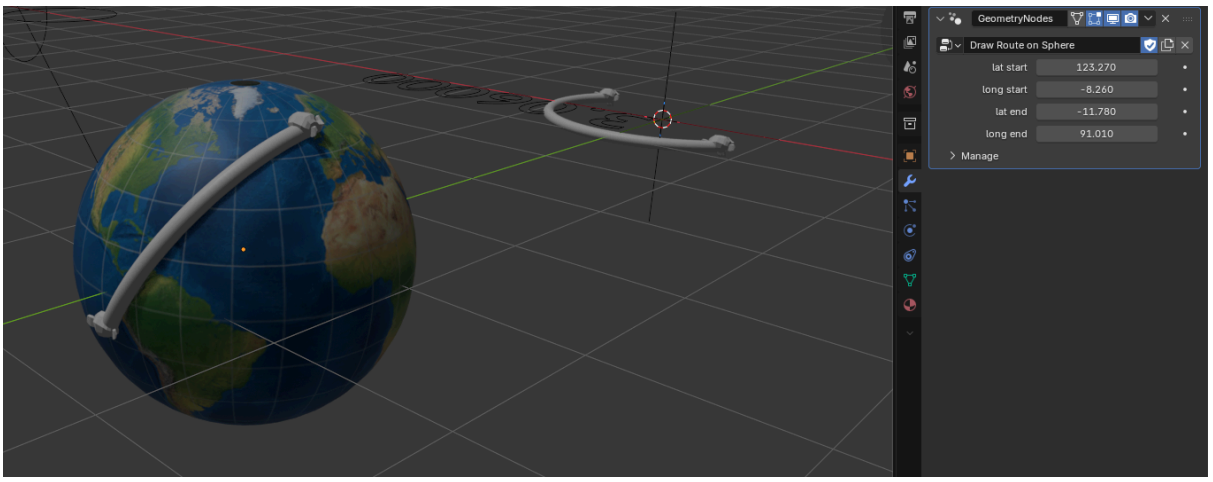


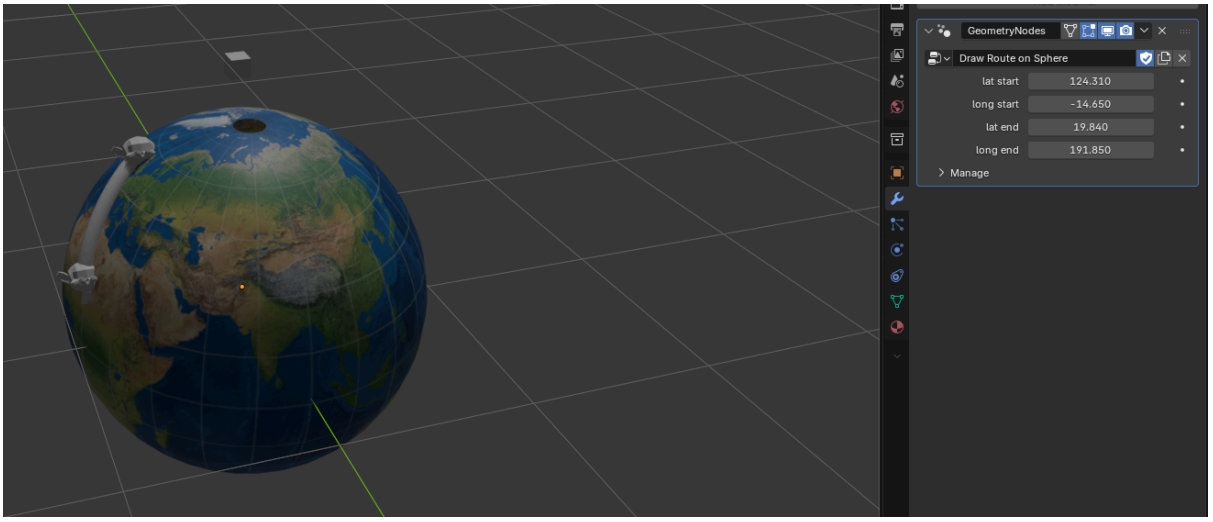
Projecting on the Globe

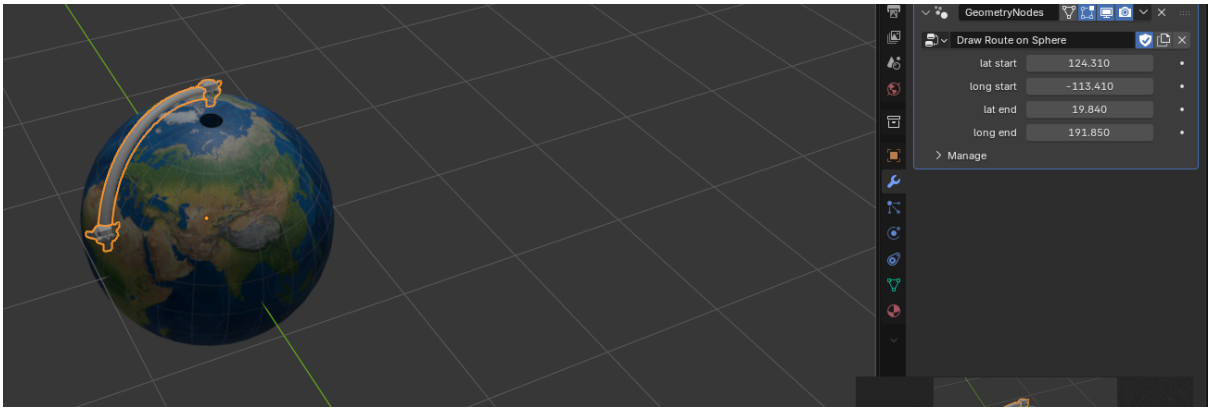
Now to put it all together, I need to:

1. Take two points defined in latitude, turn them into spherical coordinates and transform them into cartesian.
2. Rotate down to the plane using the rotation stuff I made.
3. Get their position in spherical coordinates, or rather just their longitude after transformation, since their latitude / incline will necessarily be 0.
4. Draw the line between them and get the measurement, using the stuff I just made.
5. Rotate that line back up to the sphere.

It wasn't too complicated. Did have to go back a bit to fix a problem with the function that draws a line at the equator, when the "min" and "max" (which I renamed, more aptly, to start and end) where "flipped" i.e. start was larger than end, it broke, but I fixed it. And now:







A fully node based tool to draw a great circle line between any two points on the globe, and measure its length! (the length measurement isn't implemented as a number being displayed but the length has to be known to draw the line, and I can just multiply that by the radius of the planet in mathematical terms (it currently isn't earth sized in blender) to get it, so no worries)
Also not quite "any" it specifically breaks when the initial longitude is exactly zero, but all other edge cases should be covered and that one should be quite easy to fix for the final version.

Projecting back to the plane

With this done, I just have to make a new operation which projects things from the globe down to the Mercator. Took some figuring stuff out, including having to manually configure a parameter R to fit the existing map. But finally, after all this work, I've done it.

A line drawn between two points, specifically Moscow and New York.

Did You Know..??

Shortest distance between two places on Earth is not a straight line..!!

And the same line connecting New York and Moscow, used as a common visual representation of how certain curved lines, on a mercator projection, are actually shorter than straight ones.

Final touches

Since I’m writing a report on this work, I’m going back and doing some final touches, really just amounting to cleaning up and slightly restructuring how things are done.